asio多线程模式IOThreadPool

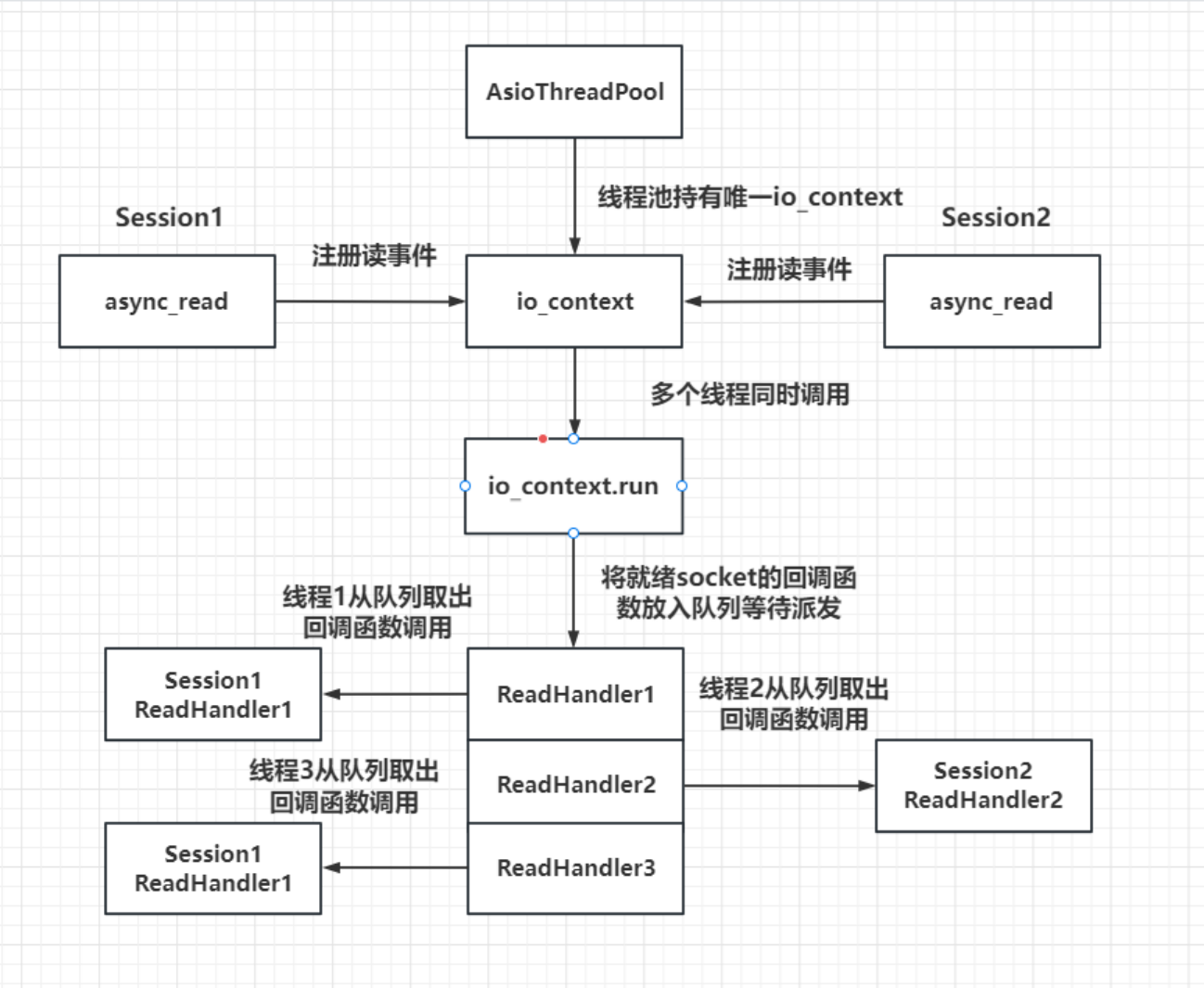
Posted on 2023-06-11 08:29:24 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2R2LjXaJqlXvTqHLY4034ojk5pa)

简介

今天给大家介绍asio多线程模式的第二种，之前我们介绍了IOServicePool的方式，一个IOServicePool开启n个线程和n个iocontext，每个线程内独立运行iocontext, 各个iocontext监听各自绑定的socket是否就绪，如果就绪就在各自线程里触发回调函数。为避免线程安全问题，我们将网络数据封装为逻辑包投递给逻辑系统，逻辑系统有一个单独线程处理，这样将网络IO和逻辑处理解耦合，极大的提高了服务器IO层面的吞吐率。这一次介绍的另一种多线程模式IOThreadPool，我们只初始化一个iocontext用来监听服务器的读写事件，包括新连接到来的监听也用这个iocontext。只是我们让 `iocontext.run` 在多个线程中调用，这样回调函数就会被不同的线程触发，从这个角度看回调函数被并发调用了。

结构图

线程池模式的多线程模型调度结构图,如下



先实现IOThreadPool

```
#include <boost/asio.hpp>
#include "Singleton.h"
class AsioThreadPool:public Singleton<AsioThreadPool>
{
public:
    friend class Singleton<AsioThreadPool>;
    ~AsioThreadPool(){}
    AsioThreadPool& operator=(const AsioThreadPool&) = delete;
    AsioThreadPool(const AsioThreadPool&) = delete;
    boost::asio::io_context& GetIOService();
    void Stop();
private:
    AsioThreadPool(int threadNum = std::thread::hardware_concurrency());
    boost::asio::io_context _service;
    std::unique_ptr<boost::asio::io_context::work> _work;
    std::vector<std::thread> _threads;

};
```

`AsioThreadPool` 继承了 `Singleton<AsioThreadPool>` ，实现了一个函数 `GetIOService` 获取 `iocontext`

接下来我们看看具体实现

```
#include "AsioThreadPool.h"

AsioThreadPool::AsioThreadPool(int threadNum ):_work(new boost::asio::io_context::work(_service)){
    for (int i = 0; i < threadNum; ++i) {
        _threads.emplace_back([this]() {
            _service.run();
            });
    }
}

boost::asio::io_context& AsioThreadPool::GetIOService() {
    return _service;
}

void AsioThreadPool::Stop() {
    _work.reset();
    for (auto& t : _threads) {
        t.join();
    }
}
```

构造函数中实现了一个线程池，线程池里每个线程都会运行 `_service.run` 函数， `_service.run` 函数内部就是从iocp或者epoll获取就绪描述符和绑定的回调函数，进而调用回调函数，因为回调函数是在不同的线程里调用的，所以会存在不同的线程调用同一个socket的回调函数的情况。

`_service.run` 内部在Linux环境下调用的是 `epoll_wait` 返回所有就绪的描述符列表，在windows上会循环调用 `GetQueuedCompletionStatus` 函数返回就绪的描述符，二者原理类似，进而通过描述符找到对应的注册的回调函数，然后调用回调函数。

比如iocp的流程是这样的

```
IOCP的使用主要分为以下几步：

1 创建完成端口(iocp)对象
2 创建一个或多个工作线程，在完成端口上执行并处理投递到完成端口上的I/O请求
3 Socket关联iocp对象，在Socket上投递网络事件
4 工作线程调用GetQueuedCompletionStatus函数获取完成通知封包，取得事件信息并进行处理
```

epoll的流程是这样的

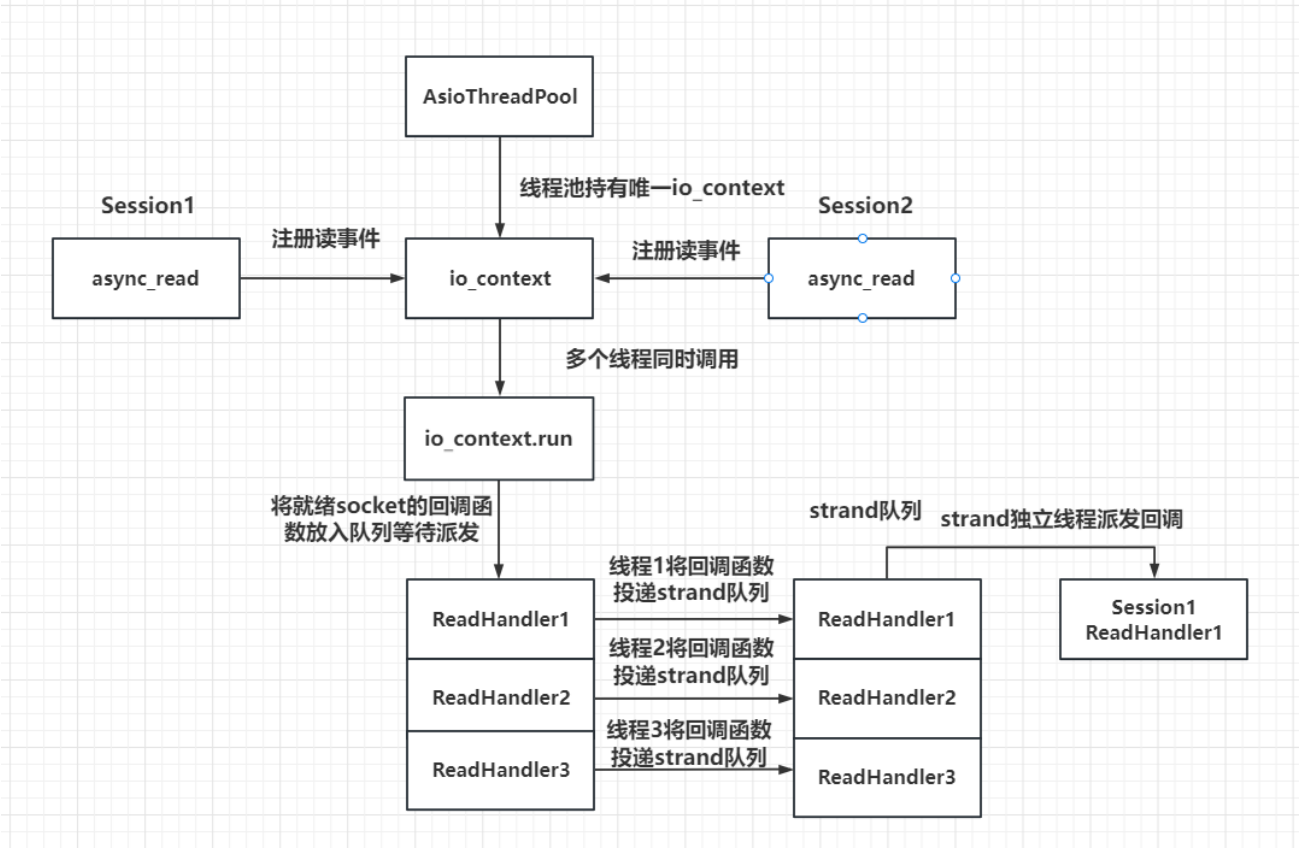
1. 1 调用epoll_creat在内核中创建一张epoll表
2. 2 开辟一片包含n个epoll_event大小的连续空间
3. 3 将要监听的socket注册到epoll表里
4. 4 调用epoll_wait，传入之前我们开辟的连续空间，epoll_wait返回就绪的epoll_event列表，epoll会将就绪的socket信息写入我们之前开辟的连续空间

隐患

IOThreadPool模式有一个隐患，同一个socket的就绪后，触发的回调函数可能在不同的线程里，比如第一次是在线程1，第二次是在线程3，如果这两次触发间隔时间不大，那么很可能出现不同线程并发访问数据的情况，比如在处理读事件时，第一次回调触发后我们从socket的接收缓冲区读数据出来，第二次回调触发,还是从socket的接收缓冲区读数据，就会造成两个线程同时从socket中读数据的情况，会造成数据混乱。

利用strand改进

对于多线程触发回调函数的情况，我们可以利用asio提供的串行类strand封装一下，这样就可以被串行调用了，其基本原理就是在线程各自调用函数时取消了直接调用的方式，而是利用一个strand类型的对象将要调用的函数投递到strand管理的队列中，再由一个统一的线程调用回调函数，调用是串行的，解决了线程并发带来的安全问题。



图中当socket就绪后并不是由多个线程调用每个socket注册的回调函数，而是将回调函数投递给strand管理的队列，再由strand统一调度派发。

为了让回调函数被派发到strand的队列，我们只需要在注册回调函数时加一层strand的包装即可。

在CSession类中添加一个成员变量

```
1. strand<io_context::executor_type> _strand;
```

CSession的构造函数

```
1. CSession::CSession(boost::asio::io_context& io_context, CServer* server):
2.     _socket(io_context), _server(server), _b_close(false),
3.     _b_head_parse(false), _strand(io_context.get_executor()){
4.     boost::uuids::uuid  a_uuid = boost::uuids::random_generator()();
5.     _uuid = boost::uuids::to_string(a_uuid);
6.     _recv_head_node = make_shared<MsgNode>(HEAD_TOTAL_LEN);
7. }
```

可以看到_strand的初始化是放在初始化列表里，利用 `io_context.get_executor()` 返回的执行

器构造strand。

因为在asio中无论iocontext还是strand，底层都是通过executor调度的，我们将他理解为调度器就可以了，如果多个iocontext和strand的调度器是一个，那他们的消息派发统一由这个调度器执行。

我们利用iocontext的调度器构造strand，这样他们统一由一个调度器管理。在绑定回调函数的调度器时，我们选择strand绑定即可。

比如我们在Start函数里添加绑定 ，将回调函数的调用者绑定为 `_strand`

```
1. void CSession::Start(){
2.     ::memset(_data, 0, MAX_LENGTH);
3. 
4.     _socket.async_read_some(boost::asio::buffer(_data, MAX_LENGTH),
5.         boost::asio::bind_executor(_strand, std::bind(&CSession::HandleRead, this,
6.             std::placeholders::_1, std::placeholders::_2, SharedSelf())));
7. }
```

同样的道理，在所有收发的地方，都将调度器绑定为 `_strand` ， 比如发送部分我们需要修改为如下

```
1.     auto& msgnode = _send_que.front();
2.     boost::asio::async_write(_socket, boost::asio::buffer(msgnode->_data, msgnode->_total_le
   n),
3.     boost::asio::bind_executor(_strand, std::bind(&CSession::HandleWrite, this, std::placehol
   ders::_1, SharedSelf()))
4.         );
```

回调函数的处理部分也做对应的修改即可。

性能对比

为了比较两种服务器多线程模式的性能，我们还是利用之前测试的客户端，客户端每隔10ms建立一个连接，总共建立100个连接，每个连接收发500次，总计10万个数据包，测试一下性能。

客户端测试代码如下

```
#include <iostream>
#include <boost/asio.hpp>
#include <thread>
#include <json/json.h>
#include <json/value.h>
#include <json/reader.h>
#include <chrono>

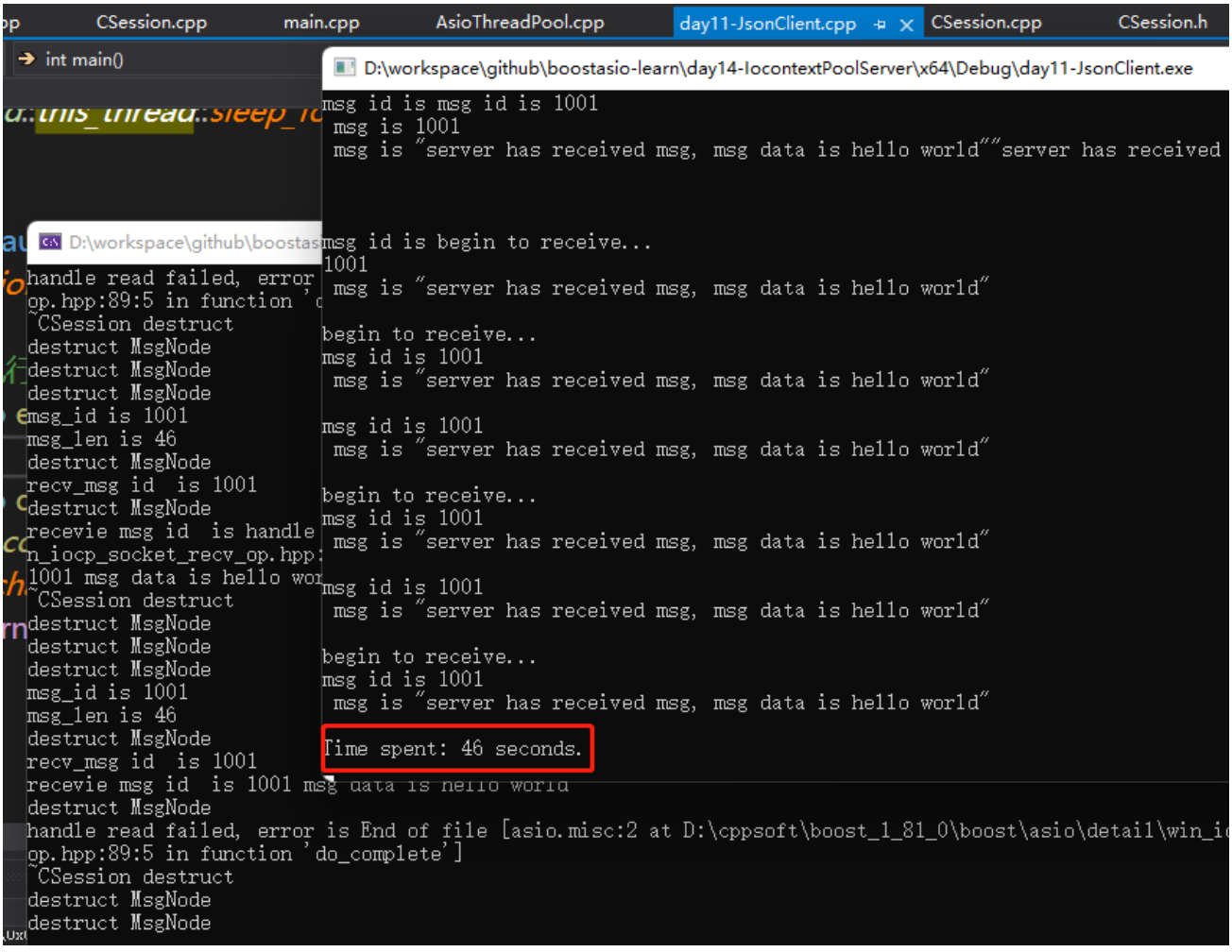
using namespace std;
using namespace boost::asio::ip;
const int MAX_LENGTH = 1024 * 2;
const int HEAD_LENGTH = 2;
const int HEAD_TOTAL = 4;

std::vector<thread> vec_threads;
int main()
{
    auto start = std::chrono::high_resolution_clock::now(); // 获取开始时间
    for (int i = 0; i < 100; i++) {
        vec_threads.emplace_back([]() {
            try {
                //创建上下文服务
                boost::asio::io_context  ioc;
                //构造endpoint
                tcp::endpoint  remote_ep(address::from_string("127.0.0.1"), 10086);
                tcp::socket  sock(ioc);
                boost::system::error_code  error = boost::asio::error::host_not_found; ;
                sock.connect(remote_ep, error);
                if (error) {
                    cout << "connect failed, code is " << error.value() << " error msg is " << error.mess
age();
                    return 0;
                }
                int i = 0;
                while (i < 500) {
                    Json::Value root;
                    root["id"] = 1001;
                    root["data"] = "hello world";
                    std::string request = root.toStyledString();
                    size_t request_length = request.length();
                    char send_data[MAX_LENGTH] = { 0 };
                    int msgid = 1001;
```

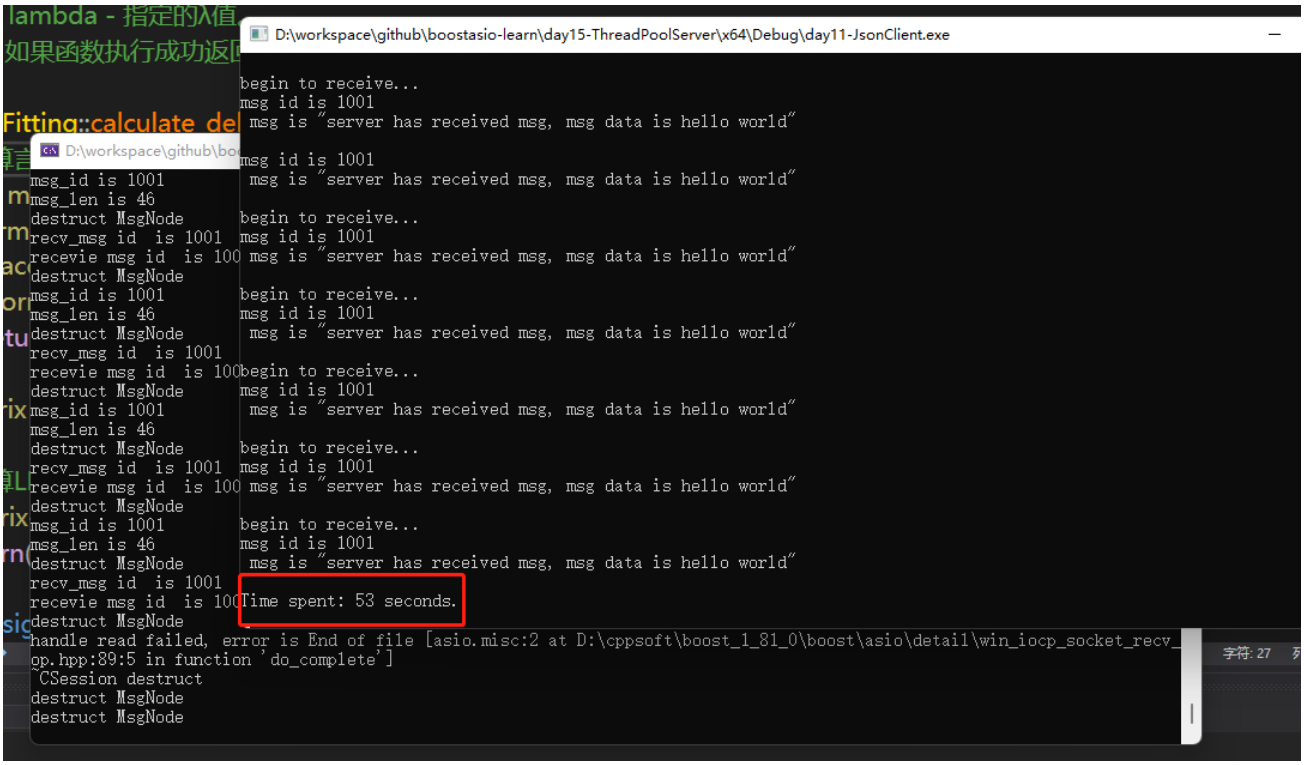
```
42.                 int msgid_host = boost::asio::detail::socket_ops::host_to_network_short(msgid);
43.                 memcpy(send_data, &msgid_host, 2);
44.                 //转为网络字节序
45.                 int request_host_length = boost::asio::detail::socket_ops::host_to_network_shor
    t(request_length);
46.                 memcpy(send_data + 2, &request_host_length, 2);
47.                 memcpy(send_data + 4, request.c_str(), request_length);
48.                 boost::asio::write(sock, boost::asio::buffer(send_data, request_length + 4));
49.                 cout << "begin to receive..." << endl;
50.
51.                 char reply_head[HEAD_TOTAL];
52.                 size_t reply_length = boost::asio::read(sock, boost::asio::buffer(reply_head, HE
    AD_TOTAL));
53.
54.                 msgid = 0;
55.                 memcpy(&msgid, reply_head, HEAD_LENGTH);
56.                 short msglen = 0;
57.                 memcpy(&msglen, reply_head + 2, HEAD_LENGTH);
58.                 //转为本地字节序
59.                 msglen = boost::asio::detail::socket_ops::network_to_host_short(msglen);
60.                 msgid = boost::asio::detail::socket_ops::network_to_host_short(msgid);
61.                 char msg[MAX_LENGTH] = { 0 };
62.                 size_t  msg_length = boost::asio::read(sock, boost::asio::buffer(msg, msglen));
63.                 Json::Reader reader;
64.                 reader.parse(std::string(msg, msg_length), root);
65.                 std::cout << "msg id is " << root["id"] << " msg is " << root["data"] << endl;
66.                 i++;
67.             }
68.         }
69.         catch (std::exception& e) {
70.             std::cerr << "Exception: " << e.what() << endl;
71.         }
72.         });
73.     std::this_thread::sleep_for(std::chrono::milliseconds(10));
74.   }
75.
76.   for (auto& t : vec_threads) {
77.     t.join();
78.   }
79.   // 执行一些需要计时的操作
80.   auto end = std::chrono::high_resolution_clock::now(); // 获取结束时间
81.
```

```
82.     auto duration = std::chrono::duration_cast<std::chrono::seconds>(end - start); // 计算时间差，单位为微秒
83.     std::cout << "Time spent: " << duration.count() << " seconds." << std::endl; // 输
84.     getchar();
85.     return 0;
86. }
```

我们先启动之前实现的AsioIOServicePool多线程模式的服务器测试,10万个数据包收发完成总计46秒



接下来我们启动ASIOThreadPool多线程模式的服务器测试，10万个数据包收发完成总计53秒



可以看出今天实现的多线程模式较之前的IOServicePool版本慢了7秒

取舍

实际的生产和开发中，我们尽可能利用C++特性，使用多核的优势，将iocontext分布在不同的线程中效率更可取一点，但也要防止线程过多导致cpu切换带来的时间片开销，所以尽量让开辟的线程数小于或等于cpu的核数，从而利用多核优势。

视频连接https://space.bilibili.com/271469206/channel/collectiondetail?sid=313101 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=313101)

源码链接https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(3181) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) 喜欢(594) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

Treen

2023-08-08 21:36:01

我的电脑，100个线程，每个线程收发1万个包，共花费1秒😎

喜欢(1) (JavaScript:void(0);) 回复(1) (JavaScript:void(0);)

共有1条回复, 点击查看

没有更多评论了,点击收起 (JavaScript:void(0);)

热门文章

slice介绍和使用

喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

喜欢(587) 浏览(1595)

解密定时器的实现细节

喜欢(566) 浏览(1449)

Linux环境搭建和编码

喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)